

SecureVault

A technical overview of how the password manager works

Temidayo Peters · Version 3 of the application · September 2026

SecureVault is a local, offline password manager written in C# with Windows Forms on .NET 8. It keeps a person's website logins in one encrypted file on their own computer, unlocked by a single master password. Nothing is sent anywhere. It was built as A-Level Computer Science coursework and is published at github.com/Temi-Peters/SecureVault, with the full coursework report alongside the code.

This document explains what the application actually does, step by step, and where each step lives in the source. It is written from the code rather than from the original design, so it also records the limitations found on reading the code back, including one serious flaw in how the encryption key is stored. Everything stated here can be checked against the named file and method.

1. The shape of the application

There are five windows and three helper classes. The windows handle what the user sees; the helpers hold the logic that matters for security, kept apart from the interface code so it can be read and tested on its own.

File	Role
Program.cs	Entry point. Shows the login window first; only if login succeeds does it open the main vault window, passing it the derived encryption key.
LoginForm.cs	Creates the master password on first run. Verifies it on every run after that. Applies the lockout.
MainForm.cs	The vault. Add, view and delete entries. Reads and writes the encrypted vault file.
ViewPasswordForm.cs	Shows one entry. Password masked by default, timed reveal, copy to clipboard with automatic clearing.
ConfirmPasswordForm.cs	Asks for the master password again before an entry can be viewed.
ChangePasswordForm.cs	Changes the master password after verifying the old one.
PasswordUtils.cs	All the cryptography: salt generation, PBKDF2, AES-256 encrypt and decrypt, the constant-time comparison, and reading and writing master.dat.
LockoutManager.cs	The exponential backoff after failed logins, persisted to lockout.dat.
SecurityPolicy.cs	The master password complexity rules.

2. The pipeline, end to end

Everything below follows the order a user experiences it. The names in brackets are the methods involved.

Step 1. First run: creating the master password

When the application starts, `LoginForm` checks whether `master.dat` exists (`PasswordUtils.LoadMasterHash`). If it does not, whatever the user types becomes the new master password, provided it passes the policy in `SecurityPolicy.ValidatePasswordComplexity`: at least 10 characters, at least one uppercase letter, one lowercase letter, one digit and one symbol, and it must not contain *password*, *12345* or *qwerty*. If the policy rejects it, the exact rule that failed is shown and nothing is saved.

If it passes, three things happen in `PasswordUtils`:

- **A salt is generated** (`GenerateSalt`): 16 random bytes from the operating system's cryptographic random number generator. The salt is not secret. Its job is to make this vault's derived values unique, so that two people who happen to choose the same master password get completely different results, and so that nobody can precompute a table of answers in advance.
- **The password is run through PBKDF2** (`HashPassword`): PBKDF2 with HMAC-SHA256, the salt, 100,000 iterations, producing 32 bytes. Section 3 explains what the iterations are for.
- **Salt and result are written to master.dat** (`SaveMasterHash`). The master password itself is never written anywhere.

The user is then asked to log in with the password they just created, rather than being let straight in.

Step 2. Every later run: verifying the master password

Before anything else, `LoginForm` asks `LockoutManager.GetRemainingLockoutSeconds` whether a lockout is in force. If it is, the attempt is refused with the remaining wait shown, and the password is not even checked.

Otherwise the salt and stored value are read back from `master.dat`, the typed password is run through the identical PBKDF2 computation, and the two 32-byte results are compared with `FixedTimeEquals`. That comparison deliberately checks every byte even after it has found a mismatch, so that the time it takes cannot reveal how many leading bytes were right.

On a wrong password, `LockoutManager.RegisterFailedAttempt` increments a failure count, records the time, and saves both to `lockout.dat`. The wait before the next attempt is 5 seconds multiplied by 2 to the power of (failures minus 1): 5, 10, 20, 40, 80, 160, then capped at 300 seconds. Because it is saved to disk, closing and reopening the application does not reset it.

On a correct password the failure count is reset to zero, and `DeriveEncryptionKey` derives the 32-byte AES key that will be used for the vault. This key exists only in memory, and is handed to `MainForm` when it opens.

The flaw in this step. `HashPassword` and `DeriveEncryptionKey` perform exactly the same computation with exactly the same inputs, and PBKDF2 is deterministic. So the 32 bytes stored in `master.dat` to verify the password are the AES key. Anyone who obtains that file together with the vault file can decrypt the vault without ever guessing the master password. The fix is described in section 6.

Step 3. Adding an entry

`MainForm.btnAdd_Click` takes the site, username and password from the three text boxes. The site must match a URL pattern (`IsValidSite`), and site and password cannot be blank; username is optional. The three are joined into one string, `site | username | password`, and added to an in-memory list.

Then `SavePasswords` rewrites the whole vault file. Each entry is encrypted on its own with `PasswordUtils.EncryptString`: AES-256 in CBC mode, using the key from login, with a fresh random 16-byte initialisation vector generated for that entry. The IV is written in front of the ciphertext. Encrypting entries separately means a corrupted entry does not take the others with it, and the fresh IV means two identical entries produce different bytes on disk.

Step 4. Opening the vault

`MainForm.LoadPasswords` runs when the vault window opens. It reads the entry count, then each length-prefixed ciphertext, and decrypts each with `DecryptBytes`, which strips the IV off the front and uses it with the key. Only the site name of each entry is shown in the list on screen; usernames and passwords stay in memory. If the file is truncated or unreadable the user is warned and the file is reset rather than the application crashing.

Step 5. Viewing an entry

Clicking View does not show anything yet. It opens `ConfirmPasswordForm`, which asks for the master password again and checks it against `master.dat` in the same way as login. Only on success does `ViewPasswordForm` open, and it does several things to limit exposure:

- The password field is read-only, keyboard shortcuts are disabled on it, and it is masked by default.
- **Show** reveals the password and starts a 10-second timer. When it fires, the field masks itself again. Clicking Show again restarts the timer.
- If the window loses focus for any reason, the field re-masks immediately (`OnDeactivate`).
- **Copy Password** puts the password on the clipboard and starts a 15-second timer. When it fires, the clipboard is cleared, but only if it still contains what was copied. If the user copied something else in the meantime, that is left alone.
- **Close** blanks the field and stops both timers before the window shuts (`SecureCleanup`).

Step 6. Deleting an entry and changing the master password

Delete asks for confirmation, removes the entry from the in-memory list, and rewrites the vault file. Change Master Password verifies the old password, applies the policy to the new one, generates a new salt, runs PBKDF2 on the new password, and overwrites `master.dat`. It does not touch the vault file, which is the second limitation in section 6.

3. What the 100,000 iterations are actually doing

A master password is a short string a person can remember. An AES-256 key is 32 bytes of data that should look random. Key derivation is the function that turns one into the other, and the design question is how expensive that function should be to run.

A single SHA-256 would produce 32 bytes, but SHA-256 is built to be fast: a high-end consumer graphics card computes on the order of 20 billion of them per second. If the key were a single hash of the password, an attacker holding the vault could try candidate passwords at that rate.

PBKDF2 feeds the hash back into itself 100,000 times. Each iteration is an HMAC, which is two SHA-256 operations, so deriving one key costs roughly 200,000 hash operations. The legitimate user pays that once at login, around a tenth of a second. An attacker pays it on every guess. The same graphics card drops from about 20 billion guesses per second to about 100,000.

What that does and does not buy. PBKDF2 multiplies the strength the password already has. It does not add any. Two worked examples, comparing 20 billion guesses per second without PBKDF2 against 100,000 with it:

Master password	Realistic entropy	Without PBKDF2	With 100,000 iterations
A human-chosen password that scrapes past the policy	about 35 bits	under 2 seconds	about 4 days
10 genuinely random printable characters	about 65 bits	about 95 years on one card	millions of years

Same algorithm, same iteration count, and the outcome is decided largely by the password. That is the honest summary: the iterations multiply the time by the same factor either way, so they cannot rescue a weak password, and for a strong one they are what turns a determined attacker's decades into something no amount of hardware makes practical.

For context, OWASP's current recommendation for PBKDF2-HMAC-SHA256 is 600,000 iterations, six times the figure used here. It was a defensible number when the project was written and the general

guidance is that it should rise over time, which is one reason the count ought to be stored with the hash rather than fixed in the code (limitation 3).

PBKDF2 was chosen over Argon2id, which is generally preferred today, for three stated reasons: it is NIST-approved with FIPS 140-2 validated implementations, it is built into .NET with no external dependency, and it gave an acceptable login time. OWASP's own guidance names FIPS validation as the case where PBKDF2 should be preferred, so the choice sits inside that carve-out.

4. The three files on disk

All three are created in the application's working directory the first time they are needed, and all three are plain binary, written with a `BinaryWriter`. Integers are little-endian 32-bit unless stated.

File	Layout	Size
master.dat	int32 salt length (16) 16-byte salt int32 hash length (32) 32-byte PBKDF2 output	56 bytes
passwords.dat	int32 entry count, then for each entry: int32 length 16-byte IV AES-256-CBC ciphertext (a multiple of 16 bytes)	4 + per entry
lockout.dat	int32 failed attempt count int64 timestamp of the last failure (.NET ticks, UTC)	12 bytes

The iteration count, the hash algorithm and the AES mode are not recorded in any file. They are fixed in the code. That is convenient and it is also why the parameters cannot be changed later without breaking existing vaults.

5. What it defends against, and what it does not

Defends against, as intended:

- **Someone guessing the master password at the login screen.** The lockout doubles the wait on every failure and survives a restart. Seven wrong attempts cost about ten minutes of waiting in total, and every attempt after that costs five minutes more.
- **Shoulder surfing.** Passwords are masked by default, revealed only briefly, and re-masked the moment the window loses focus.
- **Clipboard scraping after a copy.** The clipboard is cleared 15 seconds after a password is copied.
- **Reading the vault file on its own.** Without the salt from master.dat, an attacker holding only passwords.dat has no way to derive the key.
- **Casual inspection of the master file.** The master password itself is never stored, only a value derived from it.

Does not defend against:

- **Someone who copies both files.** Because of the flaw in section 2, step 2, the key is in master.dat. This is the case the PBKDF2 cost was meant to cover, and in the current version it does not.
- **Malware running on the same computer.** Anything with access to the process can read the decrypted entries from memory, watch the keyboard for the master password, or read the clipboard inside the 15-second window. No local password manager defends against this; it is outside what the design attempts.
- **Memory inspection after use.** Decrypted passwords are .NET strings, which cannot be wiped and remain in memory until garbage collection.

6. Known limitations, in order of seriousness

These were found by reading the code back after it was written. Each names the file, what is wrong, and the fix.

Limitation	Where	Detail and fix
1. The stored verifier is the encryption key.	PasswordUtils.cs, LoginForm.cs	HashPassword and DeriveEncryptionKey are the same PBKDF2 computation, so the 32 bytes in master.dat are the AES key. An attacker with both files decrypts the vault with no guessing. Fix: store a hash of the derived key, for example SHA-256(key), as the verifier. Login then derives the key, hashes it, and compares the hash. Recovering the key from master.dat would once again require guessing the password through PBKDF2. This changes the file format, so existing vaults need a one-off migration.
2. Changing the master password orphans the vault.	ChangePasswordForm.cs	A new salt and verifier are written, but passwords.dat is still encrypted under the old key and will not decrypt at the next login. Fix: decrypt all entries with the old key before the change and re-encrypt them with the new one, in the same operation.
3. The iteration count is not stored.	PasswordUtils.cs	100,000 exists only as a default parameter. Raising it would break every existing vault. Fix: write the count into master.dat alongside the salt, read it back at login, and re-derive at the new count on the next successful login when the stored count is lower.
4. The rotating session key does nothing.	MainForm.cs	A new sessionKey is derived every 30 seconds and its fingerprint logged, but every encrypt and decrypt uses masterKey. Fix: either remove it or give it a purpose, such as re-encrypting the in-memory entry list so decrypted data is not held under a long-lived key.
5. A pipe character corrupts an entry.	MainForm.cs	Entries are stored as site username password and split on the pipe when read, so a password containing one is cut short when viewed. Fix: a structured format such as length-prefixed fields, or escaping the delimiter.
6. The constant-time comparison is now redundant.	PasswordUtils.cs	It was written when the project targeted .NET Framework 4.7.2, which lacked one. .NET 8 provides CryptographicOperations.FixedTimeEquals. Fix: use the framework method.
7. A label is wrong.	ViewPasswordForm.cs	The screen says the clipboard clears in 10 seconds; the timer is 15. The timer was lengthened after usability testing and the label was missed. Fix: derive the label text from the timer interval.
8. Secrets live in strings.	ViewPasswordForm.cs, MainForm.cs	.NET strings are immutable and cannot be zeroed. Fix: hold decrypted values in char or byte arrays and clear them explicitly when done.

7. Parameters at a glance

Parameter	Value	Set in
Salt length	16 bytes, from RandomNumberGenerator	PasswordUtils.GenerateSalt
Key derivation	PBKDF2, HMAC-SHA256, 100,000 iterations, 32 bytes out	PasswordUtils.HashPassword PasswordUtils.DeriveEncryptionKey
Vault encryption	AES-256, CBC mode, PKCS7 padding, random 16-byte IV per entry	PasswordUtils.EncryptString
Lockout	5 s after first failure, doubling, capped at 300 s, persisted	LockoutManager
Password policy	10+ chars, 1+ upper, 1+ lower, 1+ digit, 1+ symbol, common-substring block	SecurityPolicy
Reveal timer	10 seconds	ViewPasswordForm
Clipboard clear timer	15 seconds, conditional on contents unchanged	ViewPasswordForm
Session key rotation	30 seconds (derived, not used)	MainForm
Framework	.NET 8, Windows Forms, C# 12	SecureVault.csproj

Prepared from the Version 3 source code as published in the repository. Where this document and the original coursework report differ, this document describes the code as it is.